

Multimodal Large Model+video Understanding

Function Description

After the program runs, input video-related questions through the terminal. The large model will first record a video, then parse the video content with the question, and finally reply with the answer to the question. You can also first record a video of specified duration, then ask the large model questions related to the video based on the video content.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking up the module, give a video recording command, reference the following example:

```
Record a 20 seconds video
```

After the buzzer beeps once,

```
[asr-5] [INFO] [1775201269.036976523] [asr]: -
[asr-5] [INFO] [1775201270.835488877] [asr]: XunFei ASR Finish.
[asr-5] [INFO] [1775201270.836123714] [asr]: Record a 20 seconds video.
[asr-5] [INFO] [1775201270.836783532] [asr]: @okay, let me think for a moment...
[model_service-3] [INFO] [1775201278.987083290] [model_service]: Decision making AI planning:1. Call the 'record_video' function, pass in the parameter "20", and record a 20-second video.
[model_service-3] [INFO] [1775201282.063814184] [model_service]: "json_str": {"response": "Okay, I'm going to record a 20-second video now. Let's see what happens!", "action": ["record_video(20)"]}
[model_service-3] [INFO] [1775201282.064469446] [model_service]: "action": ['record_video(20)'], "response": Okay, I'm going to record a 20-second video now. Let's see what happens!
[action_service-4] [INFO] [1775201282.072589173] [action_service]: "len(actions)": 1
[action_service-4] [INFO] [1775201282.073058176] [action_service]: "actions": ['record_video(20)']
[action_service-4] [INFO] [1775201320.716077268] [action_service]: Published message: Robot feedback: Execution record_video() completed
[action_service-4] [INFO] [1775201320.716981124] [action_service]: Checking...
[action_service-4] [INFO] [1775201320.729934728] [action_service]: pid num:[713]
[action_service-4] [INFO] [1775201322.747178486] [action_service]: sleep pids:[1, 709]
[model_service-3] [INFO] [1775201323.183670923] [model_service]: "json_str": {"response": "The video has been recorded successfully! I'm ready to help with anything else you'd like to do.", "action": ["finishtask()"]}
[model_service-3] [INFO] [1775201323.184483168] [model_service]: "action": ['finishtask()'], "response": The video has been recorded successfully! I'm ready to help with anything else you'd like to do.
[action_service-4] [INFO] [1775201323.194199311] [action_service]: "len(actions)": 1
[action_service-4] [INFO] [1775201323.194760629] [action_service]: "actions": ['finishtask()']
```

After the buzzer beeps once, recording ends. Then, based on the video content, wake up the voice module and say:

```
Did you see the red block in the video I just recorded?
```

The question here should be asked based on the actual recorded video content.

```
[asr-5] [INFO] [1775202281.800717075] [asr]: -
[asr-5] [INFO] [1775202281.808207242] [asr]: -
[asr-5] [INFO] [1775202283.726617223] [asr]: XunFei ASR Finish.
[asr-5] [INFO] [1775202283.727064576] [asr]: Did you see the red block in the video I just recorded?
[asr-5] [INFO] [1775202283.727426391] [asr]: @okay, let me think for a moment...
[model_service-3] [INFO] [1775202284.316867405] [model_service]: "json_str": {"response": "I see the red block is still there, but something seems to be obstructing the view-maybe a hand or object passed in front of the camera. Let me try to get a clearer shot!", "action": ["seewhat()"]}
[model_service-3] [INFO] [1775202284.317452684] [model_service]: "action": ['seewhat()'], "response": I see the red block is still there, but something seems to be obstructing the view-maybe a hand or object passed in front of the camera. Let me try to get a clearer shot!
[action_service-4] [INFO] [1775202284.326387586] [action_service]: "len(actions)": 1
[action_service-4] [INFO] [1775202284.327124180] [action_service]: "actions": ['seewhat()']
[model_service-3] [INFO] [1775202294.554542388] [model_service]: "json_str": {"response": "I've reviewed the video you recorded, and I can confirm that the red block was indeed visible in it. It appeared near the monitor on the desk. Let me know if you'd like me to do anything else with it!", "action": ["finishtask()"]}
[model_service-3] [INFO] [1775202294.554542723] [model_service]: "action": ['finishtask()'], "response": I've reviewed the video you recorded, and I can confirm that the red block was indeed visible in it. It appeared near the monitor on the desk. Let me know if you'd like me to do anything else with it!
[action_service-4] [INFO] [1775202294.560504379] [action_service]: "len(actions)": 1
[action_service-4] [INFO] [1775202294.561081676] [action_service]: "actions": ['finishtask()']
```

Core Code Analysis

Please refer to the **Core Code Analysis** content in the tutorial **【AI Model-Text Version】 - 【Multimodal Large Model + video Understanding】**. The voice version and text version have the same action functions; only the input method for task commands is different.

Combine With Robotic Arm

We can also combine the robotic arm with video understanding to develop new玩法. For example, prepare several opaque paper cups. After the video starts, place an object under one of the opaque paper cups, then slowly change the positions of the paper cups. After the video recording ends, ask the large model which cup the object is finally under, and then have the robotic arm point to that cup. To test this function, after waking up the voice module, input by voice:

Let's play a game. First record a video, then guess which cup has the red block at the end of the video?

The program will record a video, then analyze which cup contains the red block, and finally control the robotic arm to point to the cup with the hidden duck.

The task planning should be:

1. Call the ``record_video`` function, pass in the parameter "20", and record a 20-second video;
2. Call the ``video_understanding`` function to analyze the recorded 20-second video, and determine which cup has the red block hidden under it at the end of the video;
3. Call the ``point_to(x1, y1, x2, y2)`` function to point to the cup with the red block, where `x1, y1, x2, y2` represent the outer frame coordinates of the cup with the red block at the end of the video.